

Ex.No.1:

Develop a portfolio website for yourself which gives details about yourself for a potential recruiter.

Aim:

To develop a comprehensive and visually appealing portfolio website showcasing the skills, experience, and achievements, in order to effectively giving introduction to potential recruiters and employers.

Algorithm:

Step 1: Set Up the Project Folder Create a new folder for your project. Inside this folder, create separate subfolders for HTML, CSS, and JavaScript files.

Step 2: Create the HTML Structure Inside the HTML folder, create an index.html file and start building the basic structure of the portfolio website.

Step 3: Add CSS Styling in the CSS folder, create a styles.css file and start adding styles to the website. Customize the colors, fonts, layout, and other visual elements to match the design preferences.

Step 4: Build Sections Add different sections to the portfolio, such as "Home," "About," "Achievements", "Portfolio," and "Contact." Use HTML's <section> element to define these sections in the index.html file.

Step 5: Style Each Section Apply CSS styles to each section to make them visually appealing. CSS classes and IDs can be used to target specific elements within each section and style them accordingly.

Step 6: Add Content Fill in the content for each section. Write a brief introduction about yourself in the "About" section and showcase the projects or work in the "Portfolio" section.

Step 7: Implement Smooth Scrolling (Optional), To enhance the user experience, smooth scrolling functionality can be added using JavaScript. This will make navigation between sections smoother and more visually appealing.

Step 8: Create JavaScript Interactions (Optional), interactive elements can be added to your portfolio using JavaScript.

Step 9: Test and Debug Preview the portfolio website in different web browsers to ensure that it looks and functions as intended. Test it on various screen sizes to ensure responsiveness.

Source Code:

index.html:

```
<!DOCTYPE html>

<html>

<head>

  <meta charset="UTF-8">

  <title>Your Name - Portfolio</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

  <header>

    <h1>Your Name</h1>

    <nav class="horizontal-nav">

      <ul>

        <li><a href="#About">About</a></li>

        <li><a href="#portfolio">Projects</a></li>

        <li><a href="#resume">Resume</a></li>

        <li><a href="#contact">Contact</a></li>

      </ul>

    </nav>

  </header>
```

```
<section id="About" class="resume">
```

```
<h2>About Me</h2>
```

```

```

```
<p>Hello, I'm <strong>Your Name</strong>. I am a passionate <em>web developer</em>
with a strong foundation in front-end and back-end technologies. I have a keen interest in creating
beautiful and functional websites that deliver a great user experience.</p>
```

```
<p>In my free time, I enjoy coding, exploring the latest web development trends, and sharing
my knowledge with the community. Feel free to browse through my portfolio to see some of my
recent projects.</p>
```

```
</section>
```

```
<section id="portfolio" class="resume">
```

```
<!-- Add your portfolio items here -->
```

```
<h2>Project Details</h2>
```

```
<div >
```

```

```

```
<h3>Project 1</h3>
```

```
<p>Description of Project 1 goes here.</p>
```

```
<a href="project1-details.html">View Details</a>
```

```
</div>
```

```
<div >
```

```

```

```
<h3>Project 2</h3>
```

```
<p>Description of Project 2 goes here.</p>
```

View Details

</div>

</section>

<div id="resume">

<section class="resume">

<h2>Resume</h2>

<p>Click the button below to download my resume:</p>

Download Resume

</section>

</div>

<section id="contact" class="resume">

<h2>Contact Information</h2>

<div class="contact-info">

<p>Email: your.email@example.com</p>

<p>Phone: (123) 456-7890</p>

<p>LinkedIn: Your LinkedIn Profile</p>

<p>GitHub: Your GitHub Profile</p>

</div>

</section>

```
<footer>
```

```
<!-- Add footer information if necessary -->
```

```
</footer>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

Style.css:

```
/* Add your CSS styles here */
```

```
body {
```

```
    font-family: Arial, sans-serif;
```

```
    background-color: #f4f4f4;
```

```
    margin: 0;
```

```
    padding: 0;
```

```
}
```

```
header {
```

```
    background-color: #333;
```

```
    color: #fff;
```

```
    text-align: center;
```

```
    padding: 20px;
```

```
}
```

```
h1 {
```

```
    margin: 0;
```

```
}
```

```
.resume {
```

```
    background-color: #fff;
```

```
    padding: 20px;
```

```
    margin: 20px;
```

```
    border-radius: 5px;
```

```
}
```

```
a {
```

```
    display: inline-block;
```

```
    padding: 10px 20px;
```

```
    background-color: #333;
```

```
    color: #fff;
```

```
    text-decoration: none;
```

```
    border-radius: 5px;
```

```
}
```

```
a:hover {
```

```
background-color: #555;  
  
}
```

```
.horizontal-nav {  
  
background-color: #333;  
  
}
```

```
.horizontal-nav ul {  
  
list-style: none;  
  
margin: 0;  
  
padding: 0;  
  
}
```

```
.horizontal-nav li {  
  
display: inline;  
  
margin-right: 20px;  
  
}
```

```
.horizontal-nav a {  
  
text-decoration: none;  
  
color: #fff;  
  
font-weight: bold;
```

```
padding: 10px;
```

```
}
```

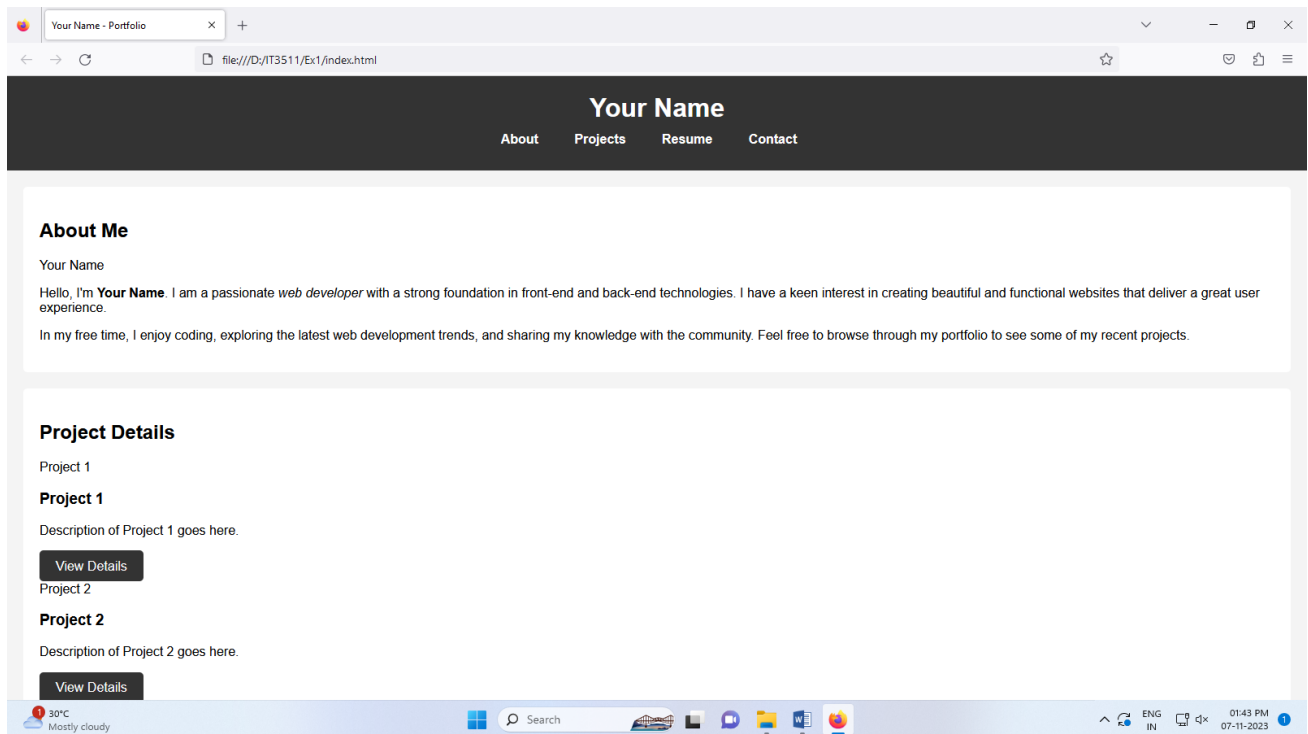
```
.horizontal-nav a:hover {
```

```
background-color: #555;
```

```
border-radius: 5px;
```

```
}
```

Output:



Result:

Thus, the portfolio website, a professional and engaging online platform that effectively communicates the skills, experience, and potential to potential recruiters was created successfully.

Ex.No.2

Create a web application to manage the TO-DO list of users, where users can login and manage their to-do items

Aim: To create a simple TO-DO list application that provide users with a basic tool to add tasks, delete tasks, and display the list of tasks.

Algorithm:

1. Initialization:

- o Initialize an empty list or array to store tasks.

2. Add Task:

- o Prompt the user to input task details (e.g., task name).
- o Create a new task object with the provided details.
- o Append the task object to the list of tasks.

3. Delete Task:

- o Display the list of tasks with a unique identifier (e.g., task number).
- o Prompt the user to enter the identifier of the task they want to delete.
- o Remove the task with the corresponding identifier from the list.

4. Display Tasks:

- o Iterate through the list of tasks.
- o Display each task's details, such as task name and identifier.

5. User Interaction Loop:

- o Implement a loop that continuously prompts the user for actions (add, delete, display, exit).
- o Based on the user's choice, perform the corresponding action (add, delete, display).

6. Exit:

- o Provide an option for the user to exit the application.

Source Code:

Html Code:

```
<html>
  <head>
    <link rel="stylesheet" href="styles.css">
  </head>
  <body>
    <div class="container">
      <div class="todo-app">
        <h2>To-Do List </h2>
        <div class="row">
          <input type="text" id="input-box" placeholder="Add your text">
          <button onclick="addTask()">Add</button>
        </div>
        <ul id="list-container">
        </ul>
      </div>
    </div>
    <script src="script.js"></script>
  </body>
</html>
```

CSS code:

```
*{
  margin:0;
  padding:0;
  font-family:'poppins',sans-serif;
  box-sizing:border-box;
}
.container{
  width:100%;
  min-height:100vh;
  background:linear-gradient(135deg,#e0341d,#30c261);
  padding:10px;
```

```
}  
.todo-app{  
  width:100%;  
  max-width:540px;  
  background:#fff;  
  margin:100px auto 20px;  
  padding: 40px 30px 70px;  
  border-radius: 10px;  
}  
.todo-app h2{  
  color:#650011;  
  display:flex;  
  align-items:center;  
  margin-bottom: 20px;  
}  
.todo-app h2 img{  
  width:30px;  
  margin-left: 10px;  
}  
.row{  
  display:flex;  
  align-items:center;  
  justify-content: space-between;  
  background: #edeef0;  
  border-radius:30px;  
  padding-left:20px;  
  margin-bottom: 25px;  
}  
input{  
  flex:1;  
  border:none;  
  outline:none;  
  background:transparent;  
  padding:10px;  
  font-weight:14px;
```

```

}
button{
    border:none;
    outline:none;
    padding:16px 50px;
    background:#ff5945;
    color:#fff;
    font-size:16px;
    cursor:pointer;
    border-radius:40px;
}
ul li{
    list-style: none;
    font-size: 17px;
    padding:12px 8px 12px 50px;
    user-select: none;
    cursor: pointer;
    position: relative;
}
ul li::before{
    content: "";
    position: absolute;
    height: 28px;
    width: 28px;
    border-radius:50%;
    background-image:url(unchecked.png);
    background-size: cover;
    background-position: center;
    top:12px;
    left:8px;
}
ul li.checked{
    color:#555;
    text-decoration: line-through;
}

```

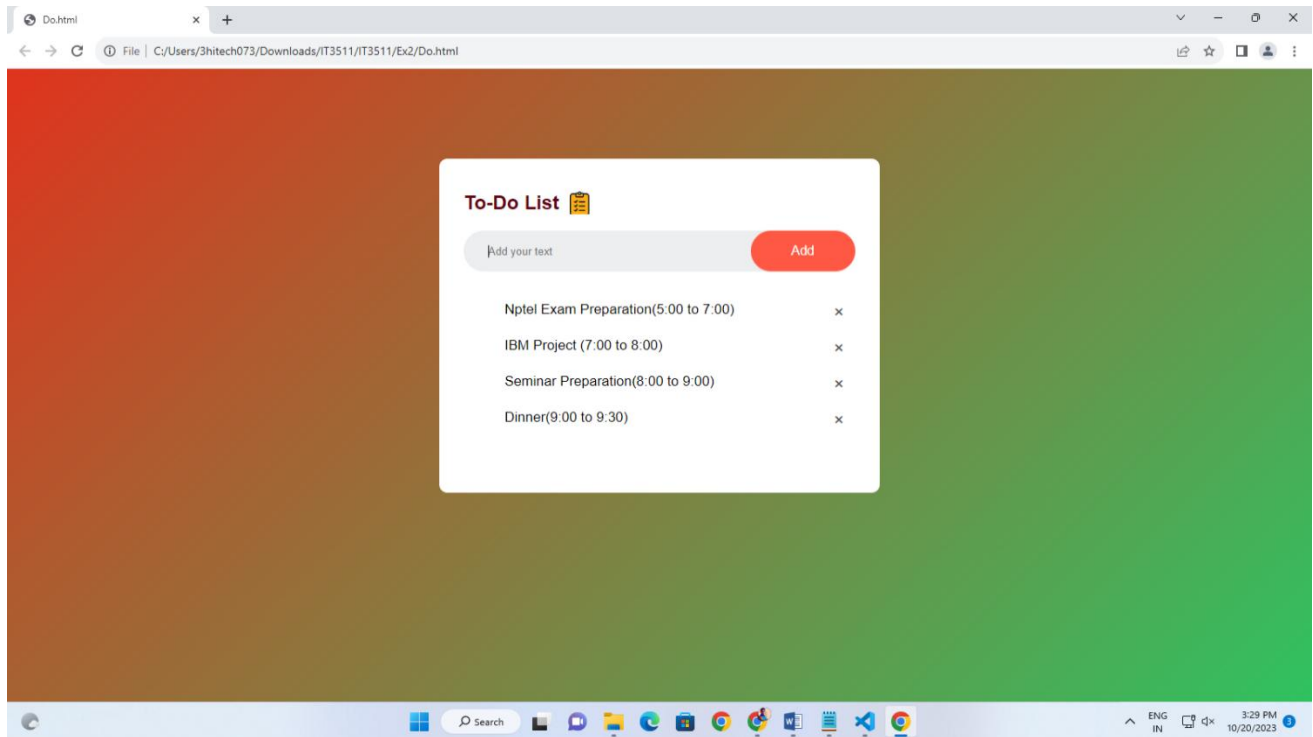
```
ul li.checked::before{
  background-image:url(checked.png);
}
ul li span{
  position: absolute;
  right: 0;
  top:5px;
  width: 40px;
  height: 40px;
  font-size: 22px;
  color:#555;
  line-height: 40px;
  text-align: center;
  border-radius: 50%;
}
ul li span:hover{
  background: #edeef0;
}
```

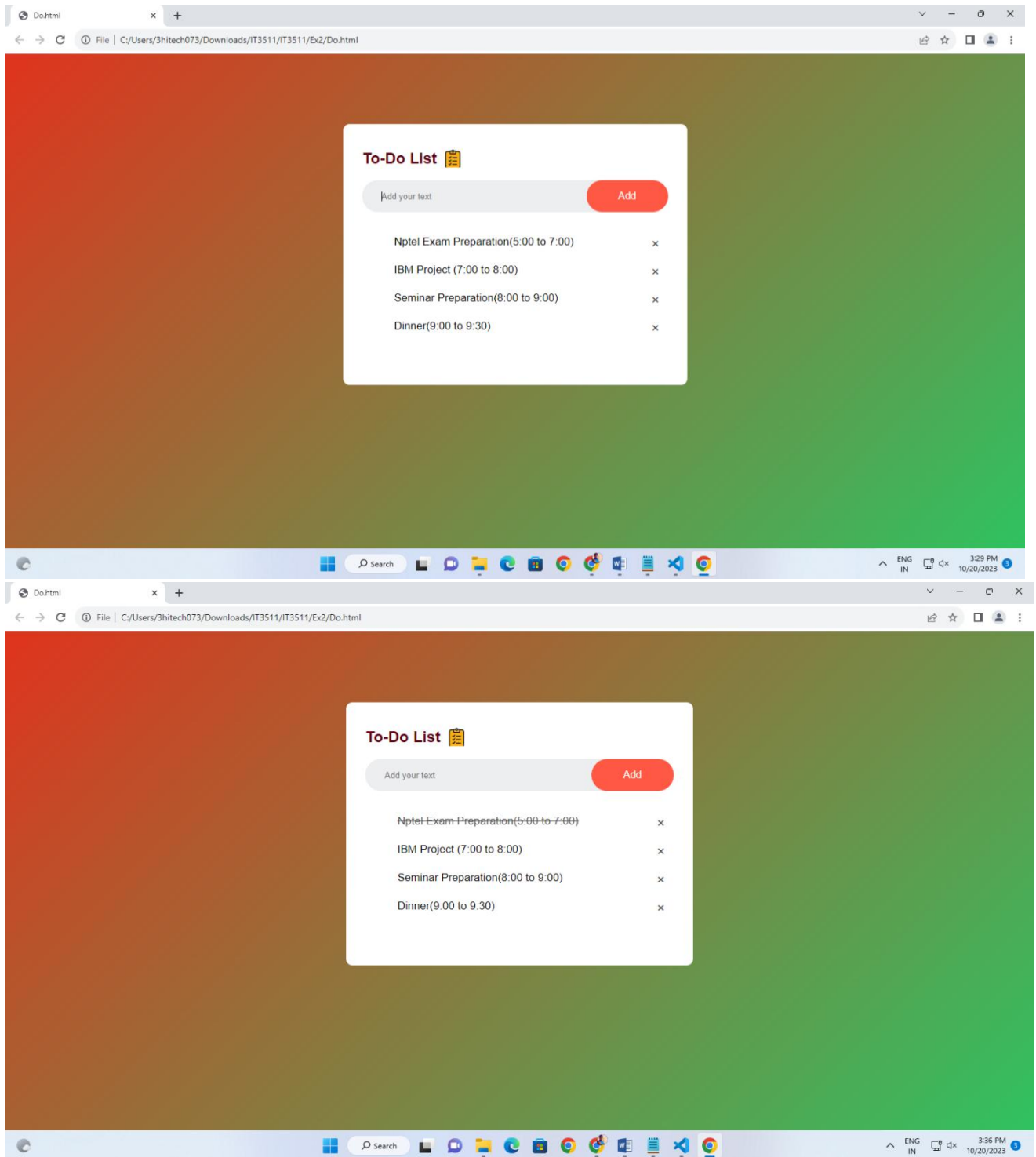
Java script Code:

```
const inputBox=document.getElementById("input-box");
const listContainer=document.getElementById("list-container");
function addTask(){
  if(inputBox.value==""){
    alert("you must write something");
  }
  else{
    let li=document.createElement("li");
    li.innerHTML=inputBox.value;
    listContainer.appendChild(li);
    let span=document.createElement("span");
    span.innerHTML="\u00d7";
    li.appendChild(span);
  }
  inputBox.value="";
```

```
        saveData();
    }
    listContainer.addEventListener("click",function(e){
        if(e.target.tagName==="LI"){
            e.target.classList.toggle("checked");
            saveData();
        }
        else if(e.target.tagName==="SPAN"){
            e.target.parentElement.remove();
            saveData();
        }
    },false);
    function saveData(){
        localStorage.setItem("data",listContainer.innerHTML);
    }
```

Output:





Result:

Thus the simple TO-DO list application with a basic tool to add tasks, delete tasks, and display the list of tasks was developed successfully.

Ex.No.3:

Create a simple micro blogging application (like twitter) that allows people to post their content which can be viewed by people who follow them.

Aim:

To create a simple micro blogging application (like twitter) that allows people to post their content which can be viewed by them.

Algorithm:

1.User Registration and Authentication

- Users register and log in using their credentials.
- Sessions or tokens are used to maintain user authentication.

2.Create a New Post (Tweet):

- Authenticated users can create new tweets.
- Tweets consist of content, user information, and a timestamp.
- Tweets are saved in a database with user references.

3.View the Post(Tweet):

- users can view the tweets with some details.

Source code:

Project Structure:

```
/microblog-app
├─ node_modules/
├─ models/
│   └─ Post.js
├─ public/
│   └─ styles.css
├─ views/
│   └─ index.ejs
├─ .gitignore
├─ package.json
└─ server.js
```

Post.js:

```
const mongoose = require('mongoose');
const postSchema = new mongoose.Schema({
  title: String,
  content: String,
  timestamp: { type: Date, default: Date.now }
});
module.exports = mongoose.model('Post', postSchema);
```

styles.css:

```
body {
  font-family: Arial, sans-serif;
  margin: 0;
  padding: 0;
}
h1 {
  text-align: center;
  margin: 20px 0;
}
form {
  display: flex;
  flex-direction: column;
  align-items: center;
}
input[type="text"],
textarea {
  width: 80%;
  padding: 10px;
  margin: 5px 0;
}
button {
  padding: 10px;
  background-color: #007bff;
  color: white;
```

```

border: none;
cursor: pointer;
}
ul {
list-style: none;
padding: 0;
}
li {
border: 1px solid #ccc;
padding: 10px;
margin: 10px 0;
}

```

index.ejs:

```

<!DOCTYPE html>
<html>
<head>
  <title>Microblog</title>
  <link rel="stylesheet" type="text/css" href="/styles.css">
</head>
<body>
  <h1>Microblog</h1>
  <form action="/create" method="POST">
    <input type="text" name="title" placeholder="Title" required>
    <textarea name="content" placeholder="Content" required></textarea>
    <button type="submit">Create Post</button>
  </form>
  <ul>
    <% posts.forEach(post => { %>
      <li>
        <h2><%= post.title %></h2>
        <p><%= post.content %></p>
        <p><em><%= post.timestamp.toLocaleString() %></em></p>
      </li>
    <% }); %>

```

```
</ul>
</body>
</html>
```

server.js

```
const express = require('express');
const mongoose = require('mongoose');
const bodyParser = require('body-parser');
const Post = require('./models/Post');
const path = require('path');
const app = express();
const PORT = process.env.PORT || 3000;

mongoose.connect('mongodb://127.0.0.1:27017/microblog_db', { useNewUrlParser: true,
useUnifiedTopology: true });

app.set('view engine', 'ejs');

// Set up middleware
app.use(bodyParser.urlencoded({ extended: true }));
app.use(express.static(path.join(__dirname, 'public')));

// Routes
app.get('/', async (req, res) => {
  const posts = await Post.find().sort({ timestamp: -1 });
  res.render('./index1.ejs', { posts });
});

app.post('/create', async (req, res) => {
  const { title, content } = req.body;
  const newPost = new Post({ title, content });
  await newPost.save();
  res.redirect('/');
});

// Start the server
app.listen(3000, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

Execution:

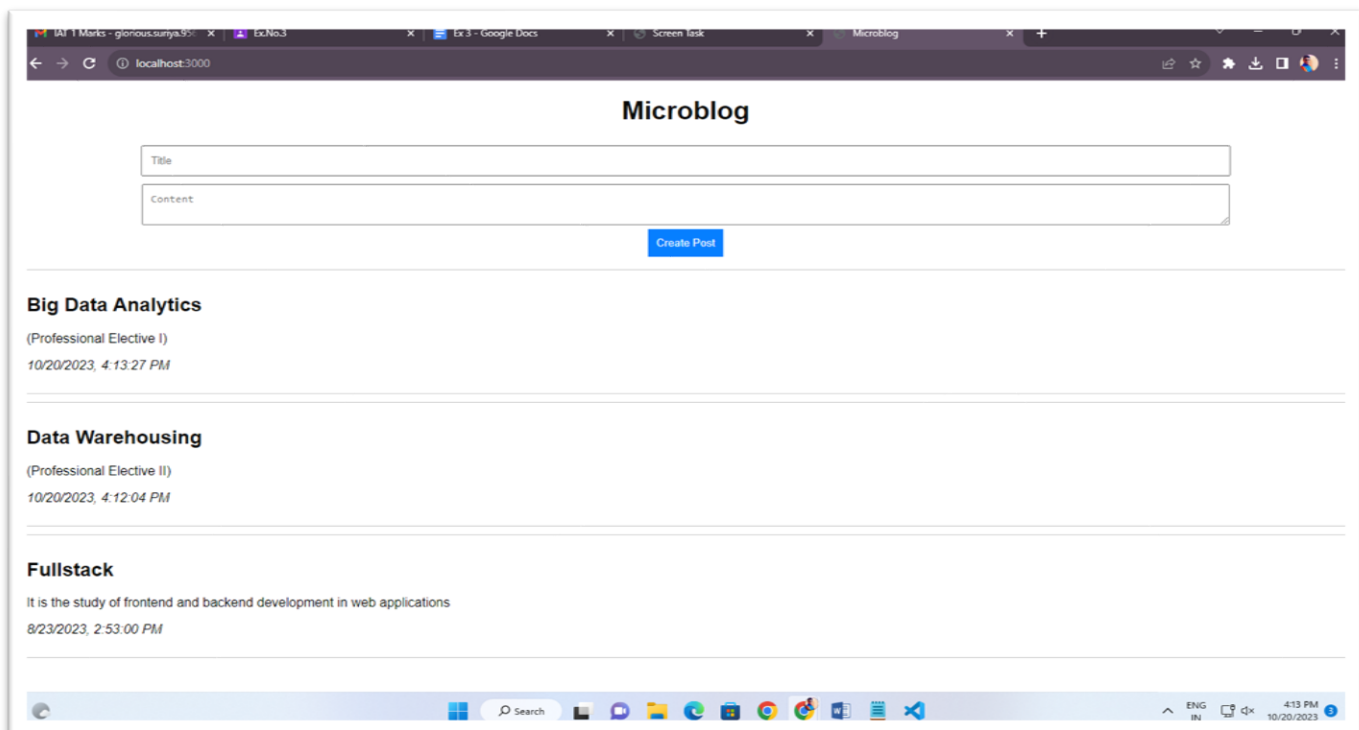
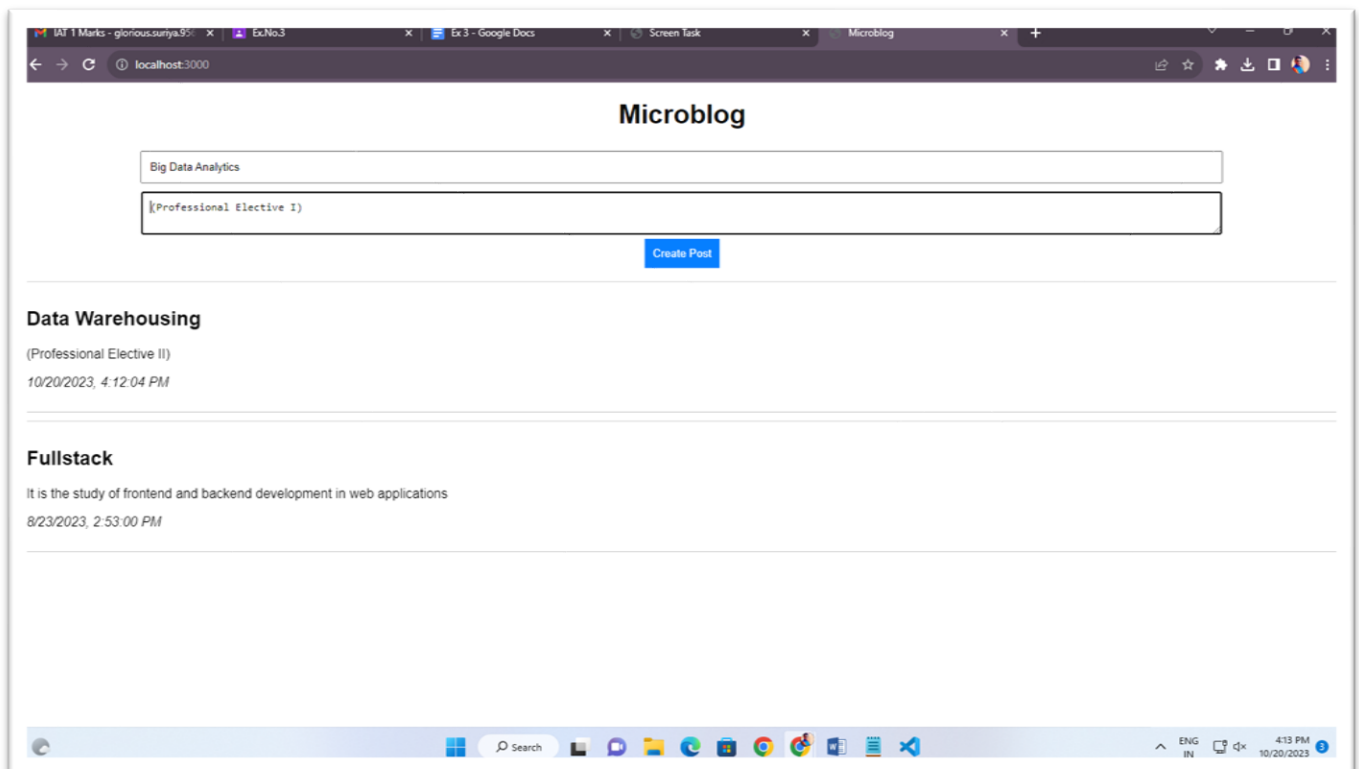
npm install express mongoose ejs body-parser

npm start

node server.js

Output:

Server is running on port 3000



Result:

Thus, a simple micro blogging application (like twitter) that allows people to post their content which can be viewed by them was created successfully.

Ex.No.4:

Create a food delivery website where users can order food from a particular restaurant listed in the website.

Aim: To create a simple food delivery website that allows users to view food items, add them to the cart, and calculate the total cost of selected items.

Algorithm:

1. **Initialization:**
 - Set up the HTML structure with the header, menu, and cart sections.
 - Link the necessary CSS stylesheet and JavaScript file.
2. **Display Menu:**
 - Create a <div> element with a class of "menu" to hold food items.
 - For each food item:
 - Create a <div> element with a class of "item".
 - Inside the item <div>:
 - Add an element for the food image.
 - Add an <h2> element for the food name.
 - Add a <p> element for the food description.
 - Add a element with a class of "price" for the food price.
 - Add a <button> element with a class of "add-to-cart" for adding to the cart.
3. **Add to Cart Interaction:**
 - Query all elements with the class "add-to-cart" using JavaScript.
 - For each "add-to-cart" button:
 - Attach a click event listener that executes the following steps:
 - Get the parent element (item) of the clicked button.
 - Extract the item name and price from the item's elements.
 - If the item is already in the cart:
 - Increment the item's count in the cart.
 - If the item is not in the cart:
 - Add the item to the cart with a count of 1.
 - Update the total cost.
 - Call the updateCart() function.
4. **Update Cart Display:**
 - Create a <div> element with a class of "cart" to hold cart contents.
 - Inside the cart <div>:
 - Add an element with an id of "cart-items" to list cart items.
 - Add an <h3> element to display the total cost.
 - Add a element with an id of "cart-total" to display the actual total.
5. **Update Cart Function (updateCart()):**
 - Clear the contents of the cart items list.
 - For each item in the cart:
 - Create a element to represent the item and its count.
 - Append the element to the "cart-items" list.
 - Update the "cart-total" span with the total cost.
6. **Styling:**
 - Add CSS styling to format the header, menu, items, cart, and buttons.
 - Style the layout, fonts, colors, and alignment to create an appealing design.
7. **Testing:**
 - Test the website's functionality by adding items to the cart and verifying the total.
 -

Source Code:

index.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="styles.css">
  <title>Food Delivery</title>
</head>
<body>
  <header>
    <h1>Food Delivery</h1>
  </header>
  <div class="menu">
    <div class="item">
      
      <h2>Burger</h2>
      <p>Delicious burger with all the fixings.</p>
      <span class="price">$243.99</span>
      <button class="add-to-cart">Add to Cart</button>
    </div>
    <div class="item">
      
      <h2>Pizza</h2>
      <p>Classic cheese pizza with tomato sauce.</p>
      <span class="price">$90.66</span>
      <button class="add-to-cart">Add to Cart</button>
    </div>
    <div class="item">
      
      <h2>Vegetable Briyani</h2>
      <p>Dish made with rice, veggies & spices.</p>
      <span class="price">$90.99</span>
      <button class="add-to-cart">Add to Cart</button>
    </div>
  </div>
</body>
</html>
```

```
</div>
<!-- Add more items here -->
</div>
<div class="cart">
  <h2>Cart</h2>
  <ul id="cart-items"></ul>
  <h3>Total: $<span id="cart-total">0.00</span></h3>
</div>
<script src="script.js"></script>
</body>
</html>
```

Styles.css

```
body {
  font-family: Arial, sans-serif;
  margin: 0;
  padding: 0;
}
header {
  background-color: #333;
  color: white;
  text-align: center;
  padding: 10px;
}
.menu {
  display: flex;
  justify-content: space-around;
  flex-wrap: wrap;
  padding: 20px;
}
.item {
  border: 1px solid #ddd;
  padding: 10px;
  margin: 10px;
  text-align: center;
```

```

    width: 200px;
  }
  .item img {
    max-width: 100%;
  }
  .price {
    font-weight: bold;
  }
  .add-to-cart {
    background-color: #333;
    color: white;
    border: none;
    padding: 5px 10px;
    cursor: pointer;
  }
  .cart {
    border: 1px solid #ddd;
    padding: 10px 10px;
    position: fixed;
    top: 550px;
    right: 40px;
    width: 250px;
  }

```

Script.js

```

const addToCartButtons = document.querySelectorAll('.add-to-cart');
const cartItemsList = document.getElementById('cart-items');
const cartTotal = document.getElementById('cart-total');
let total = 0;
const cart = {};
addToCartButtons.forEach(button => {
  button.addEventListener('click', () => {
    const item = button.parentElement;
    const itemName = item.querySelector('h2').textContent;
    const itemPrice = parseFloat(item.querySelector('.price').textContent.slice(1));
    if (cart[itemName]) {

```

```
    cart[itemName] += 1;
  } else {
    cart[itemName] = 1;
  }
  total += itemPrice;
  updateCart();
});
});

function updateCart() {
  cartItemsList.innerHTML = "";
  for (const item in cart) {
    const li = document.createElement('li');
    li.textContent = `${item} x${cart[item]}`;
    cartItemsList.appendChild(li);
  }
  cartTotal.textContent = total.toFixed(2);
}
```

Output:

Food Delivery



Burger
Delicious burger with all the fixings.
\$243.99 [Add to Cart](#)



Pizza
Classic cheese pizza with tomato sauce.
\$90.66 [Add to Cart](#)



Vegetable Briyani
Dish made with rice, veggies & spices.
\$90.99 [Add to Cart](#)

Cart
Total: \$0.00

Food Delivery



Burger
Delicious burger with all the fixings.
\$8.99 [Add to Cart](#)



Pizza
Classic cheese pizza with tomato sauce.
\$10.99 [Add to Cart](#)



Pasta
Delicious burger with all the fixings.
\$23.99 [Add to Cart](#)

Cart

- Burger x1
- Pizza x1
- Pasta x1

Total: \$43.97

Result:

Thus, a simple food delivery website to view food items, add them to the cart, and calculate the total cost of selected items was developed successfully.

Ex. No:5:

Develop a classifieds web application to buy and sell used products.

Aim:

To create a simple classified web application in Node.js where users can post, view, and purchase classified advertisements.

Algorithm:

1. Registration and Authentication:

- Allow users to register and log in securely to create and manage their classified listings.

2. Listing Creation:

- Enable users to create new classified listings.
- Each listing should include essential details like a title, description, category, price, and contact information.

3. Listing Display:

- Display classified listings in a user-friendly format.
- Allow users to view and purchase details of individual listings.

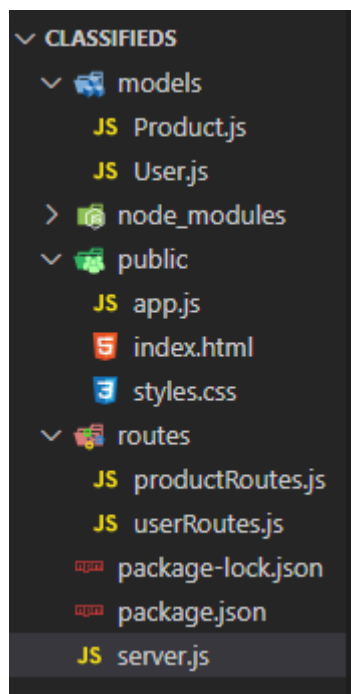
4. Data Storage:

- Use a database (e.g., MongoDB, PostgreSQL) to store classified listings and user information.
- Implement database CRUD (Create, Read, Update, Delete) operations to manage listings.

5. Error Handling:

- Include error handling mechanisms to gracefully handle exceptions and errors that may occur during the application's operation.

Project structure:



Source Code:

Product.js

```
const mongoose = require('mongoose');
const productSchema = new mongoose.Schema({
  title: String,
  description: String,
  price: Number,
  status: { type: String, enum: ['available', 'purchased'], default: 'available' }
});
const Product = mongoose.model('Product', productSchema);
module.exports = Product;
```

User.js:

```
const mongoose = require('mongoose');
const userSchema = new mongoose.Schema({
  username: String,
  email: String,
  password: String,
});
module.exports = mongoose.model('User', userSchema);
```

app.js:

```
document.addEventListener('DOMContentLoaded', () => {
  const productList = document.getElementById('product-list');
  // Function to fetch and display product listings
  const fetchProducts = () => {
    fetch('/api/products')
      .then(response => response.json())
      .then(products => {
        productList.innerHTML = ""; // Clear existing list
        products.forEach(product => {
          const li = document.createElement('li');
          li.textContent = `${product.title} - ${product.price}`;
          // Add a "Buy" button for each product
          const actionButton = document.createElement('button');
          if (product.status === 'purchased') {
```

```

        actionButton.textContent = 'Purchased';
        actionButton.disabled = true;
    } else {
        actionButton.textContent = 'Buy';
        actionButton.addEventListener('click', () => buyProduct(product._id, actionButton));
    }
    li.appendChild(actionButton);
    productList.appendChild(li);
});
})
.catch(error => console.error('Error fetching products:', error));
};

// Call the fetchProducts function initially and after creating a listing
fetchProducts();

// Create a new product listing
const createListingButton = document.getElementById('create-listing-button');
createListingButton.addEventListener('click', () => {
    const title = document.getElementById('title').value;
    const description = document.getElementById('description').value;
    const price = document.getElementById('price').value;
    const newData = {
        title: title,
        description: description,
        price: price
    };
    fetch('/api/products', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json'
        },
        body: JSON.stringify(newData)
    })
    .then(response => response.json())
    .then(product => {
        console.log('Product created:', product);
        // Clear input fields
        document.getElementById('title').value = "";

```

```

    document.getElementById('description').value = "";
    document.getElementById('price').value = "";
    // Fetch and display products after creating a listing
    fetchProducts();
  })
  .catch(error => console.error('Error creating product:', error));
});

// Buy a product
const buyProduct = (productId, actionButton) => {
  fetch(`/api/products/${productId}/buy`, {
    method: 'PUT',
    headers: {
      'Content-Type': 'application/json'
    }
  })
  .then(response => response.json())
  .then(updatedProduct => {
    console.log('Product purchased:', updatedProduct);
    // Update the button text to "Purchased" and disable it
    actionButton.textContent = 'Purchased';
    actionButton.disabled = true;
    // Optionally, you can add a CSS class to style the "Purchased" button differently
    actionButton.classList.add('purchased-button');
  })
  .catch(error => console.error('Error purchasing product:', error));
};

// Initial fetch and display of products
fetchProducts();
});

```

index.html:

```

<!DOCTYPE html>
<html>
<head>
  <title>Classifieds App</title>
  <link rel="stylesheet" type="text/css" href="styles.css">
</head>

```

```

<body>
  <h1>Welcome to Classifieds App</h1>
  <h2>Product Listings</h2>
  <ul id="product-list"></ul>
  <h2>Create a Listing</h2>
  <form id="create-listing-form">
    <label for="title">Title:</label>
    <input type="text" id="title" required><br>
    <label for="description">Description:</label>
    <textarea id="description" required></textarea><br>
    <label for="price">Price:</label>
    <input type="number" id="price" required><br>
    <button type="button" id="create-listing-button">Create Listing</button>
  </form>
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
  <script src="app.js"></script>
</body>
</html>

```

styels.css:

```

body {
  font-family: Arial, sans-serif;
  margin: 20px;
}
h1 {
  color: #333;
}
h2 {
  color: #555;
  margin-top: 20px;
}
ul {
  list-style-type: none;
  padding: 0;
}
li {
  margin: 10px 0;
  padding: 10px;
}

```

```

border: 1px solid #ddd;
background-color: #f9f9f9;
}
label {
display: inline-block;
width: 100px;
}
input, textarea {
width: 300px;
padding: 5px;
margin-bottom: 10px;
}
button {
background-color: #333;
color: #fff;
padding: 5px 10px;
border: none;
cursor: pointer;
}
/* Style for the "Purchased" button */
.purchased-button {
background-color: #ccc;
color: #888;
cursor: not-allowed;
}

```

productRoutes.js:

```

const express = require('express');
const router = express.Router();
const Product = require('../models/Product');
// Create a new product listing
router.post('/', async (req, res) => {
  try {
    const newProduct = await Product.create(req.body);
    res.json(newProduct);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
}

```

```

});
// Get all product listings
router.get('/', async (req, res) => {
  try {
    const products = await Product.find();
    res.json(products);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});
// Update product status to mark as purchased
router.put('/:productId/buy', async (req, res) => {
  try {
    const productId = req.params.productId;
    // Find the product by ID and update its status or relevant fields
    const updatedProduct = await Product.findByIdAndUpdate(
      productId,
      { $set: { status: 'purchased' } }, // Update the status or relevant fields here
      { new: true } // Return the updated product
    );
    if (!updatedProduct) {
      return res.status(404).json({ message: 'Product not found' });
    }
    res.json(updatedProduct);
  } catch (error) {
    console.error('Error buying product:', error);
    res.status(500).json({ message: 'An error occurred while buying the product' });
  }
});
module.exports = router;

```

userRoutes.js:

```

const express = require('express');
const router = express.Router();
const User = require('../models/User');
// User registration
router.post('/register', async (req, res) => {
  try {

```

```

    const newUser = await User.create(req.body);
    res.json(newUser);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});
module.exports = router;

```

server.js:

```

const express = require('express');
const mongoose = require('mongoose');
const bodyParser = require('body-parser');
const cors = require('cors');
const path = require('path'); // Import the 'path' module
const app = express();
// Middleware
app.use(cors());
app.use(bodyParser.json());
// Connect to MongoDB
mongoose.connect('mongodb://127.0.0.1:27017/classifieds', {
  useNewUrlParser: true,
  useUnifiedTopology: true,
});
mongoose.connection.on('error', console.error.bind(console, 'MongoDB connection error:'));
app.use(express.static(path.join(__dirname, 'public')));
// Routes
const userRoutes = require('./routes/userRoutes');
const productRoutes = require('./routes/productRoutes');
app.use('/api/users', userRoutes);
app.use('/api/products', productRoutes);

const PORT = process.env.PORT || 4000;
app.listen(4000, () => {
  console.log(`Server is running on port ${PORT}`);
});

```


Output:

Welcome to Classifieds App

Product Listings

hai - \$10000 Buy

Create a Listing

Title:

Description:

Price:

Create Listing

Result:

Thus, a simple classified web application in Node.js where users can post, view, and purchase classified advertisements.

Ex.No.6: Develop a leave management system for an organization where users can apply different types of leaves such as casual leave and medical leave. They also can view the available number of days.

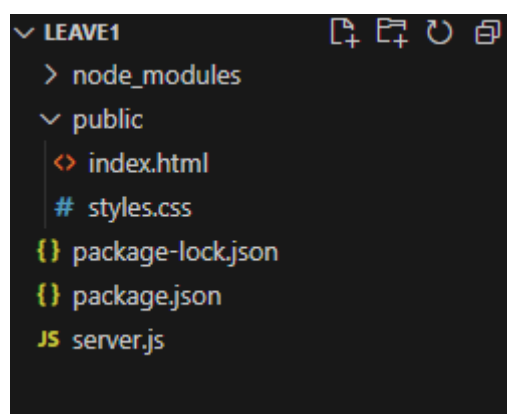
Aim:

To develop a leave management system in Node.js to provide a convenient and efficient way for employees to request and manage their leave and help administrators to process and track these requests.

Algorithm:

1. **User Authentication and Authorization:**
 - Implement user authentication and authorization to ensure that only authorized users (employees and administrators) can access the system.
2. **Database Setup:**
 - Set up a MongoDB, MySQL, or another database of your choice to store employee data, leave requests, and other relevant information.
 - Create the necessary database schemas/models to represent users and leave requests.
3. **Leave Request Creation:**
 - Provide an interface for employees to submit leave requests, specifying the start date, end date, and reason.

Project Structure:



Source code:**index.html:**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Leave Management System</title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
  <h1>Welcome to Leave Management System</h1>
  <form action="http://localhost:4000/apply" method="POST">
    <label for="employeeName">Employee Name:</label>
    <input type="text" id="employeeName" name="employeeName" required>
    <br>
    <label for="leaveType">Leave Type:</label>
    <select id="leaveType" name="leaveType" required>
      <option value="vacation">Vacation</option>
      <option value="sick">Sick Leave</option>
    </select>
    <br>
    <label for="startDate">Start Date:</label>
    <input type="date" id="startDate" name="startDate" required>
    <br>
    <label for="endDate">End Date:</label>
    <input type="date" id="endDate" name="endDate" required>
    <br>
    <button type="submit">Submit</button>
  </form>
</body>
</html>
```

styles.css:

```
body {
    font-family: Arial, sans-serif;
    text-align: center;
    margin: 50px;
}
h1 {
    color: #333;
}
form {
    margin: 20px auto;
    width: 300px;
    border: 1px solid #333;
    padding: 20px;
}
label {
    display: block;
    margin-bottom: 5px;
}
input[type="text"],
input[type="date"],
select {
    width: 100%;
    padding: 5px;
    margin-bottom: 10px;
}
button {
    background-color: #333;
    color: #fff;
    border: none;
    padding: 10px 20px;
    cursor: pointer;
}
```

server.js:

```
const express = require('express');
const bodyParser = require('body-parser');
const mongoose = require('mongoose');
//const ejs = require('ejs');
const app = express();
//app.set('view engine', 'ejs');
app.use(express.static('public'));
app.use(bodyParser.urlencoded({ extended:false }));
// Connect to MongoDB (replace with your MongoDB connection string)
mongoose.connect('mongodb://127.0.0.1:27017/leave_management_system', {
  useNewUrlParser: true,
  useUnifiedTopology: true,
});
mongoose.connection.on('open', () => {
  // Start your server or perform database operations here
});
// Define a LeaveRequest schema
const leaveRequestSchema = new mongoose.Schema({
  employeeName: String,
  leaveType: String,
  startDate: Date,
  endDate: Date,
});
// Create a LeaveRequest model
const LeaveRequest = mongoose.model('LeaveRequest', leaveRequestSchema);
app.get('/', (req, res) => {
  // Retrieve leave requests from MongoDB
  LeaveRequest.find({}, (err, leaveRequests) => {
    if (err) {
      console.error(err);
      res.status(500).send('Internal Server Error');
    } else {
      res.render('index', { leaveRequests });
    }
  })
});
```

```
});  
});  
  
app.post('/apply', (req, res) => {  
  // Create and save a new leave request  
  const newLeaveRequest = new LeaveRequest({  
    employeeName:req.body.employeeName,  
    leaveType:req.body.leaveType,  
    startDate:req.body.startDate,  
    endDate:req.body.endDate  
  });  
  newLeaveRequest.save();  
  res.end("Submitted");  
});  
  
const port = process.env.PORT || 4000; // Use the provided port or default to 3000  
app.listen(port, () => {  
  console.log(`Server is running on http://localhost:${port}`);  
});
```

Output:

Welcome to Leave Management System

Employee Name:

Leave Type:

Sick Leave

▼

Start Date:

25 - 10 - 2023

📅

End Date:

31 - 10 - 2023

📅

Submit

Result:

Thus, a leave management system in Node.js to provide a convenient and efficient way for employees to request and manage their leave was developed successfully.

Ex.No. 7 Develop a simple dashboard for project management where the statuses of various tasks are available. New tasks can be added and the status of existing tasks can be changed among Pending, InProgress or Completed.

Aim:

To develop a simple project management dashboard that allows users to efficiently manage tasks and monitor their status using Node.js and React js.

Algorithm:

1. **Task Management:** Provide a platform for users to add, view, and edit tasks for a project.
2. **Status Tracking:** Enable users to change the status of tasks among "Pending," "InProgress," or "Completed."
3. **User Interface:** Develop an intuitive and user-friendly web interface for easy interaction with tasks and project status.

Backend (Node.js):

1. Set up a Node.js server using a framework like Express.js.
2. Create a data model for tasks, including attributes like task name, description, status, and task owner.
3. Implement API routes for:
 - Retrieving all tasks.
 - Adding a new task.
 - Updating the status of a task.
4. Use a data storage mechanism (e.g., in-memory data, a database) to store and manage task data.
5. Add error handling and validation for API requests.

Frontend (React):

1. Create a React application using create-react-app or a similar tool.
2. Design the user interface with components for:
 - A task list to display existing tasks.
 - A task form for adding new tasks.
 - Buttons to update task status.
3. Use React state to manage the list of tasks and their status within the frontend.
4. Implement functionality to add new tasks and display them
5. Develop an interactive user interface for changing the status of tasks.
6. Use React state and hooks for real-time updates when a task's status changes.
7. Ensure the UI is responsive and visually appealing.

Source code:

Project Structure:

```
project-management-dashboard/  
├─ client/  
│   └─ src/  
│       └─ components/  
│           └─ TaskForm.js  
│           └─ TaskList.js  
│           └─ App.js  
└─ package.json  
server.js
```

Step 1: Set up your project Create a new directory for your project and initialize it with Node.js and React.

```
mkdir project-management-dashboard  
cd project-management-dashboard  
npx create-react-app client
```

Step 2: Create the Node.js Server

Create a simple Node.js server using Express to serve your React app and handle API requests. Create a file named server.js in your project root:

server.js

```
// server.js  
  
const express = require('express');  
const bodyParser = require('body-parser');  
const cors = require('cors');  
const app = express();  
app.use(cors());  
app.use(bodyParser.json());  
app.listen(5000, () => {  
  console.log('Server is running on port 5000');  
});
```

Step 3: Set up the React Frontend

You can create components for your dashboard in the client/src directory. For this example, we'll create a basic layout with a form to add new tasks and a list to display existing tasks.

Step 4: Create React Components

1. TaskForm.js

```
// client/src/components/TaskForm.js
```

```
import React, { useState } from 'react';

const TaskForm = ({ addTask }) => {
  const [taskName, setTaskName] = useState("");
  const handleSubmit = (e) => {
    e.preventDefault();
    if (taskName.trim() !== "") {
      addTask(taskName);
      setTaskName(""); // Clear the input field
    }
  };
  return (
    <div className="task-form">
      <h2>Add New Task</h2>
      <form onSubmit={handleSubmit}>
        <input
          type="text"
          placeholder="Task Name"
          value={taskName}
          onChange={(e) => setTaskName(e.target.value)} />
        <button type="submit">Add Task</button>
      </form>
    </div> );
};

export default TaskForm;
```

2. TaskList.js

// **client/src/components/TaskList.js**

```
import React from 'react';const TaskList = ({ tasks, updateStatus }) => {  
  return (  
    <div className="task-list">  
      <h2>Task List</h2>  
      <ul>  
        {tasks.map((task, index) => (  
          <li key={index}>  
            <span>{task.name}</span>  
            <div>  
              <span>Status: {task.status}</span>  
              <button onClick={() => updateStatus(index)}>  
                Update Status  
              </button>  
            </div>  
          </li>  
        ))}  
      </ul>  
    </div>  
  );  
};  
  
export default TaskList;
```

Step 5: App Component and State Management

Now, create an App.js file in the client/src directory to manage the application state.

// **client/src/App.js**

```
import React, { useState } from 'react';  
import TaskForm from './components/TaskForm';  
import TaskList from './components/TaskList';
```

```

function App() {
  const [tasks, setTasks] = useState([]);
  const addTask = (taskName) => {
    setTasks([...tasks, { name: taskName, status: 'Pending' }]);
  };
  const updateStatus = (index) => {
    const updatedTasks = [...tasks];
    updatedTasks[index].status =
      updatedTasks[index].status === 'Pending' ? 'InProgress' : 'Completed';
    setTasks(updatedTasks);
  };
  return (
    <div className="App">
      <TaskForm addTask={addTask} />
      <TaskList tasks={tasks} updateStatus={updateStatus} />
    </div> );
}
export default App;

```

Step 6: Connect the React App to the Server In your client/package.json, add a proxy to connect your React app to the server:

"proxy": ["http://localhost:5000"](http://localhost:5000)

Step 7: Start the Application In the root directory, start both the React frontend and the Node.js server:

Concurrent Development: To concurrently run both the server and the React app, you can use the concurrently package. Install it in your project root directory:

npm install concurrently

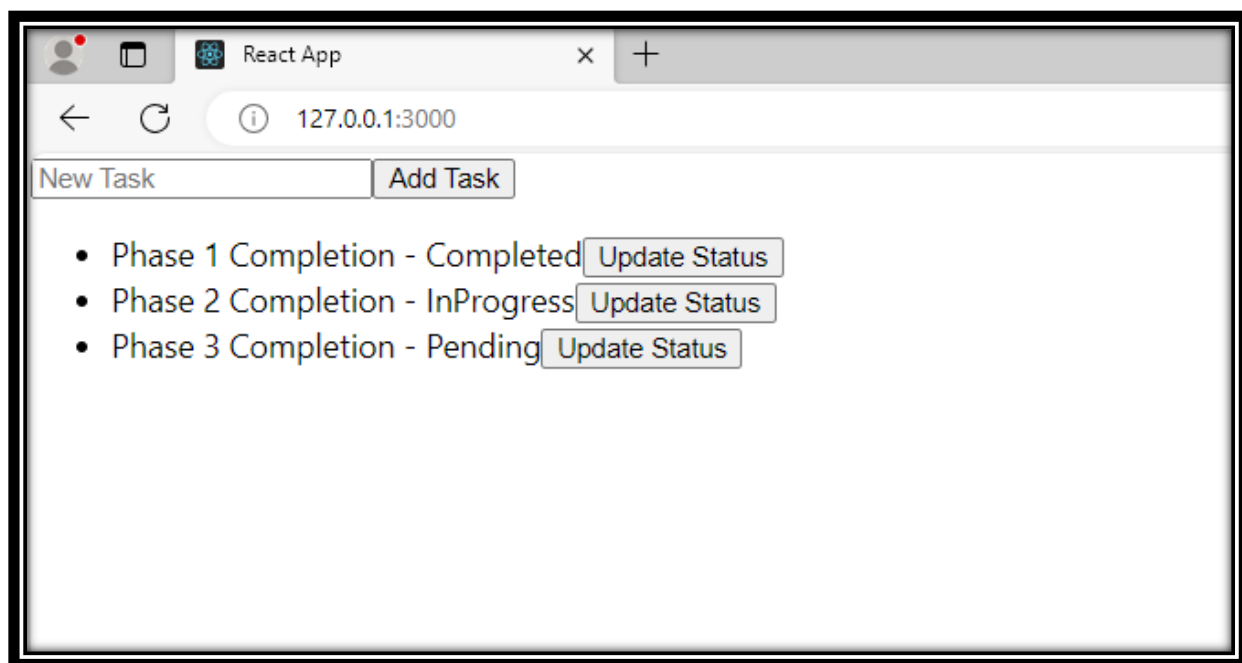
Then, update the "scripts" section in your project's package.json as follows:

```

"scripts": {
  "client": "cd client && npm start",
  "server": "node server.js",
  "start": "concurrently \"npm run server\" \"npm run client\""
}

```

Now, start both the server and the React app using npm start.



Result:

Thus,a simple classified web application in Node.js where users can post, view, and purchase classified advertisements.

Ex.No. 8 Develop an online survey application where a collection of questions is available and users are asked to answer any random 5 questions.

Aim:

To develop an online survey application where a collection of questions is available and users are asked to answer any random 5 questions using Node.js and React js.

Algorithm:

1. Initialization:

- Initialize the React application.
- Create a component named `Survey` for taking surveys.
- Create a component named `Results` for displaying survey results.

2. Survey Component:

- Initialize state variables for selected questions, survey responses, and whether to view results.
- Create a `questions` array containing survey questions.
- Render the survey form:
 - Provide a button to select random questions.
 - Display selected questions with input fields for answers.
 - Include a "Submit" button to submit responses.
- Add functionality to:
 - Select random questions for the survey.
 - Clear survey responses between persons.
 - Submit survey responses and clear input values.
 - Toggle the visibility of survey results.

3. Results Component:

- Initialize the `Results` component with a prop to receive survey responses.
- Display the survey results:
 - Iterate through the survey responses.
 - For each person, display their responses, replacing "Student" with "Person."

4. App Component:

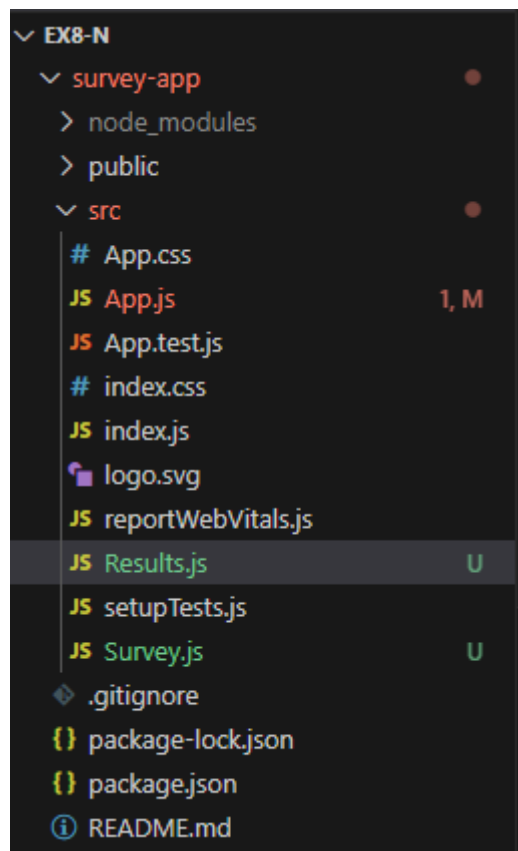
- Create the main `App` component to render the application.
- Include the `Survey` component within the `App` component.

5. Running the Application:

- Start the development server to run the application.
- Users can click "Select Random Questions" to take a survey, clear responses as needed, and view the results with the "View Results" button.

Source code:

Project Structure:



Steps:

Step 1: Set up a new React application

Set up a new React application using Create React App:

```
npx create-react-app survey-app
cd survey-app
```

Step 2: Create a Survey Component

Inside the src directory, create a Survey.js component to manage the survey functionality.

Survey.js

```
import React, { useState, useRef } from 'react';
import Results from './Results';
const questions = [
  'What is your favorite color?',
  'What is your favorite animal?',
  'Where do you live?',
```

```

    'What is your age?',
    'What is your gender?',
  ];
  const Survey = () => {
    const [selectedQuestions, setSelectedQuestions] = useState([]);
    const [surveyResponses, setSurveyResponses] = useState([]);
    const [viewResults, setViewResults] = useState(false);
    const inputRefs = useRef([]); // To store references to input elements
    const selectRandomQuestions = () => {
      const randomQuestions = [];
      while (randomQuestions.length < 5) {
        const randomIndex = Math.floor(Math.random() * questions.length);
        if (!randomQuestions.includes(randomIndex)) {
          randomQuestions.push(randomIndex);
        }
      }
      setSelectedQuestions(randomQuestions);
    };
    const clearSurvey = () => {
      inputRefs.current.forEach((inputRef) => {
        inputRef.value = ""; // Clear input values
      });
    };
    const submitSurvey = (responses) => {
      setSurveyResponses([...surveyResponses, responses]);
      clearSurvey();
    };
    const toggleViewResults = () => {
      setViewResults(!viewResults);
    };
    return (
      <div>
        <h1>Survey Questions</h1>
        {!viewResults && (
          <div>

```

```

    <button onClick={selectRandomQuestions}>Select Random Questions</button>
    <ul>
      {selectedQuestions.map((index) => (
        <li key={index}>
          {questions[index]}
          <input type="text" placeholder="Your Answer" ref={(el) => (inputRefs.current[index] =
el)} />
        </li>
      ))}
    </ul>
    <button
      onClick={() => {
        const answers = selectedQuestions.map((index) => inputRefs.current[index].value);
        submitSurvey(answers);
      }}
    >
      Submit
    </button>
  </div>
)}
<button onClick={toggleViewResults}>
  {viewResults ? 'Hide Results' : 'View Results'}
</button>
{viewResults && <Results surveyResponses={surveyResponses} />}
</div>
);
};
export default Survey;

```

Step 3: Create a Results Component

1. a viewResults state variable to control whether the survey results are visible.
2. A "View Results" button toggles the visibility of the results. When clicked, it either shows or hides the survey results.
3. The survey-taking portion of the component is only displayed when viewResults is false.

Results.js

```
import React from 'react';
```

```

const Results = ({ surveyResponses }) => {
  return (
    <div>
      <h1>Survey Results</h1>
      <ul>
        {surveyResponses.map((response, index) => (
          <li key={index}>
            <strong>Person {index + 1}</strong>
            <ul>
              {response.map((answer, questionIndex) => (
                <li key={questionIndex}>
                  {`Question ${questionIndex + 1}: ${answer}`}
                </li>
              ))}
            </ul>
          </li>
        ))}
      </ul>
    </div>
  );
};
export default Results;

```

Step 4: Update `App.js`

Ensure that you have the `Survey` component as the main content of the `App.js` file.

App.js

```

import React from 'react';
import './App.css';
import Survey from './Survey';
function App() {
  return (
    <div className="App">
      <Survey /></div>
    );}

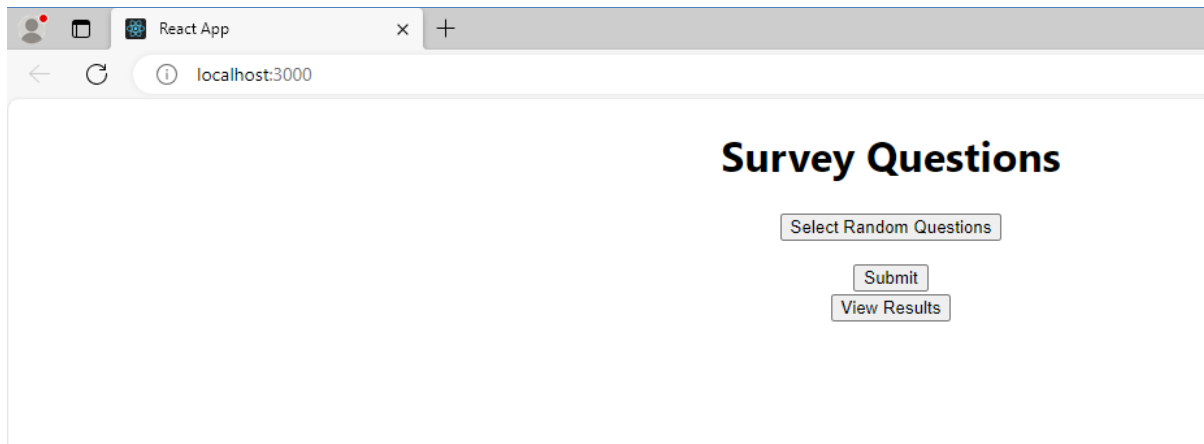
export default App;

```

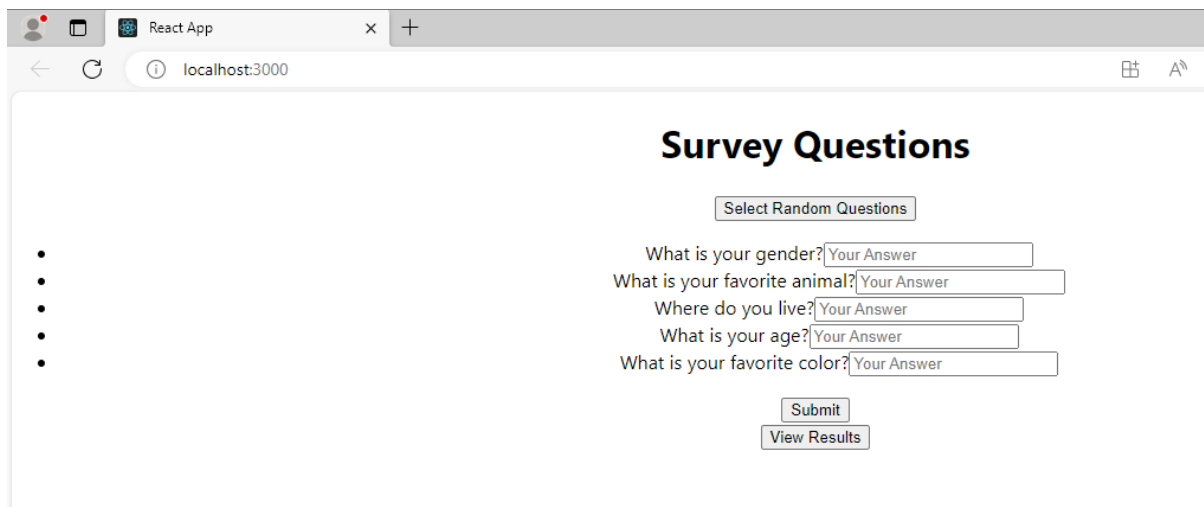
Step 4: Running the Application

You can run the application using `npm start`, and you'll be able to take the survey, submit responses, and view the survey results after submission of the survey.

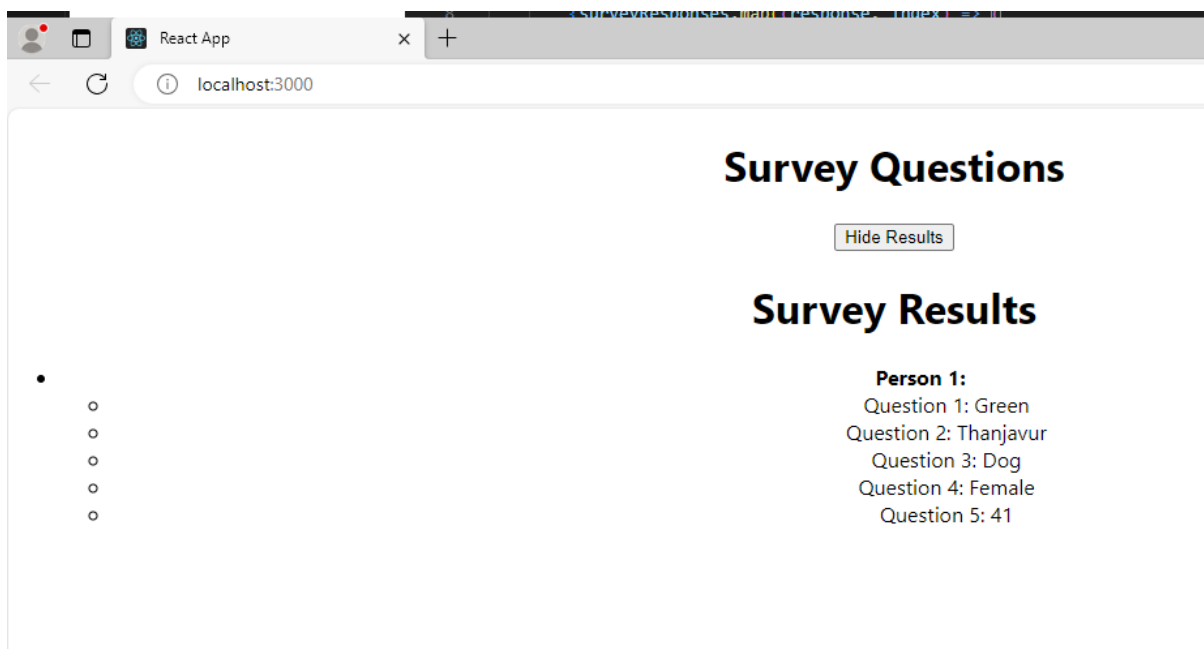
Output:



A screenshot of a web browser window titled 'React App' at 'localhost:3000'. The page displays the heading 'Survey Questions' in a large, bold, black font. Below the heading, there are three buttons stacked vertically: 'Select Random Questions', 'Submit', and 'View Results'.



A screenshot of the same web browser window. The 'Select Random Questions' button has been clicked, and the page now displays a list of five survey questions, each followed by a text input field labeled 'Your Answer'. The questions are: 'What is your gender?', 'What is your favorite animal?', 'Where do you live?', 'What is your age?', and 'What is your favorite color?'. The 'Submit' and 'View Results' buttons remain at the bottom.



A screenshot of the web browser window showing the 'Survey Results' page. The heading 'Survey Questions' is still present, but a 'Hide Results' button has appeared below it. Below this, the heading 'Survey Results' is displayed in a large, bold, black font. To the left of the results, there is a list of five radio buttons. To the right, under the heading 'Person 1:', the answers to the five questions are listed: 'Question 1: Green', 'Question 2: Thanjavur', 'Question 3: Dog', 'Question 4: Female', and 'Question 5: 41'.

Result:

Thus, an online survey application where a collection of questions is available and users are asked to answer any random 5 questions using Node.js and React js was developed successfully.

